

PROJECT REPORT ON

RESUME ANALYZER

**Submitted in partial fulfilment of the requirement for the award of
the**

**degree of
MASTER OF COMPUTER APPLICATIONS**

Submitted by:

**DHIRAJ KUMAR
University Roll No. 1103040**

Under the Guidance of

Prof. Dr. A. S. Shukla



**Department of Computer Applications
Graphic Era (Deemed to be University)**

Dehradun, Uttarakhand

May-2026

TABLE OF CONTENTS

CERTIFICATE	i
ACKNOWLEDGEMENT	ii
CERTIFICATE OF ORIGINALITY	iii
CONTENTS	iv- v
LIST OF FIGURES & ABBREVIATIONS	vi-vii
ABSTRACT	viii
CHAPTER 1: INTRODUCTION	1-5
1.1 INTRODUCTION	1
1.2 PROBLEM STATEMENT	3
1.3 OBJECTIVE	4
1.4 SIGNIFICANCE AND MOTIVATION	4
1.5 ORGANIZATION	5
CHAPTER 2: LITERATURE SURVEY	6-11
2.1 LITERATURE SURVEY	6
2.2 KEY GAPS	10
CHAPTER 3: SYSTEM DEVELOPMENT	12-25
3.1 REQUIREMENTS	12
3.2 PROJECT DESIGN AND ARCHITECTURE	16
3.3 DATA PREPERATION	20
3.4 IMPLEMENTATION	22
CHAPTER 4: CORE FEATURES & MODULES	26-28
4.1 Features	26
4.2 Parsing Module	26
4.3 Scoring Module	27
4.4 Dashboard	28

CHAPTER 5: USER INTERFACE & EXPERIENCE	29-32
5.1 DESIGN PHILOSOPHY	29
5.2 INTERFACE	29
5.3 CANDITATE INTERFACE	30
CHAPTER 6: TESTING	33-39
6.1 TESTING STRATEGY	33
6.2 TEST CASE OUTCOMES	35
6.2 TESTING STRATEGY & RESULT	38
6.3 TESTING TYPE	38
CHAPTE7: RESULT AND EVALUATION	40-44
7.1 RESUME ANALYZER	40
CHAPTER 8: CONC;USION AND FUTURE SCOPES	42-47
8.1 CONCLUSION	42
8.2 FUTURE SCOPE	47
APPENDICES	48-51
FIELD DESCRIPTION	52
REFERENCES	53-54

LIST OF FIGURES

S. No.	Title	Page No.
1	Flow of the app	15
2	Flow of the Resume Analyzer	16
3	Flow of Personality Insights	16
4	Flow of Resume Builder	17
5	Flow of Resume Matching	18
6	Environment setup	20
7	Data preparation	21
8	Prompts for LLM to execute	22
9	Resume Builder execution	23
10	Resume Matcher Implementation	23
11	Cover Letter generation	23
12	Personality Insights generator	24
13	User Interface	24
14	Result of Test Case no. 1	30
15	Result of Test Case no. 2	31
16	Result of Test Case no. 3	31
17	User uploaded resume	32
18	Details and Skills extraction from resume	33
19	Skills and course recommendation	33
20	Resume Grading and insights	34

LIST OF ABBREVIATIONS

ABBREVIATIONS	MEANING
NLP	Natural Language Processing
NLTK	Natural Language Toolkit
UI/UX	User Interface/User Experience
LLM	Large Language Model

ABSTRACT

It's becoming more and more important for job searchers to be able to expertly tailor their resumes for every application to further their careers in the fast-paced industry. The purpose of this project is to change the way resume analysis and job profile matching is done by introducing the use of the Natural Language Processing (NLP) techniques, mainly the Large Language Models (LLMs). The conventional methods of resume data extraction and job matching have used only NLP, which is the reason why they have not been able to achieve the best accuracy and diversity. The use of LLMs to improve the quality and the quantity of the insights that are extracted from the resumes, thus, the LLMs will be able to match the candidate profiles with the job requirements more precisely.

Our approach consists of several important aspects. Firstly, we use NLP to extract the necessary data from the resumes, which will guarantee a full data capture. Thus, by applying resume matching, we improve the match of job profiles and candidate can easily find jobs whose skills matches with their profiles. In addition, our application has a feature of resume builder that helps to make resumes suitable for job roles by means of the information extracted and the analysis of the personality. Through the resolution of the existing methodologies gap, our project establishes a new level of resume analysis and job matching and thus, enables the candidates to find their way to their preferred job.

In addition, the integration of Large Language Models enhances contextual understanding, enabling accurate resume scoring, job role prediction, personalized course recommendations, cover letter generation, and personality insight analysis. The application also includes a resume builder and job matching module that helps candidates optimize their resumes according to industry standards. The developed system significantly reduces manual effort, improves recruitment efficiency, minimizes human bias, and provides intelligent career guidance to job seekers. This project demonstrates the practical application of AI-driven technologies in modern recruitment and career development systems.

Furthermore, the system improves the accuracy and speed of candidate selection. It also provides a smart and user-friendly platform for both recruiters and job seekers.

CHAPTER 1

INTRODUCTION

1.1 Introduction

Particularly in the dynamic sector of career development, the innovative integration of Natural Language Processing (NLP) into the Resume Analyzer signifies a paradigm shift in the conventional practice of resume design for job applications. This Resume Analyzer is an explanation of the complex relationships between natural language processing algorithms and how it may revolutionize the hiring process by suggesting strategic certifications and identifying relevant job roles.

Using natural language processing (NLP) and open source LLM, the Resume Analyzer reimagines the resume customization process and situates it at the intersection of cutting-edge technology and professional advancement. The tool makes predictions about a candidate's intended career job using advanced natural language processing (NLP) techniques, going beyond resume analysis. It also looks at what NLP does during evaluation of resumes to gather qualifications, credentials, experience, and skills. It is based on the NLP breakthrough, an advanced paradigm in technological development, which makes it suitable for the Resume Analyzer.

Resume Analyzer employs natural language processing machine learning techniques, which provide detailed analysis of the textual data in resumes. The system shall analyze information related to the applicant's background data to find out compatible job positions. The system is accurate at arriving at the right employment function for an individual based on detailed scrutiny of job descriptions and the incorporation of industry terminologies. Now with the addition of machine learning, the Resume Analyzer is much more than a statistical tool. A prediction machine that the company claims could alter how prospective employees think about their future tracks of careers.

The Resume Analyzer is also forecasted as it promotes specific qualifications that are in line with the required job description. This system takes into consideration a number of factors relevant to the candidate's competencies available in the markets and the general state of the industry. In addition to task prediction, the objective gives users useful information and directs them toward

Certifications that might enhance their standing in the field. By positioning the Resume Analyzer as a holistic career development tool, this proactive approach fosters a culture of continuous learning and skill development.

Our approach consists of several important aspects. First, we use NLP for the extraction of necessary information from the resume, which will ensure that all data is captured. Therefore, by applying the resume matching, we improve the match of the job profiles and candidate can find jobs whose skills match their profiles easily. Besides, our application includes a feature of a resume builder that helps to make resumes suitable for job roles by means of the information extracted and analysis of personality.

The Resume Analyzer not only focuses on evaluating resumes but also aims to assist candidates in improving their professional profiles. It provides detailed feedback on resume quality, identifies missing or weak skill areas, and suggests relevant certifications or training programs that can enhance employability. This feature transforms the system into a dual-purpose platform that supports both recruiters and job seekers. While recruiters benefit from faster and more accurate candidate shortlisting, candidates gain valuable insights to improve their chances of selection.

In addition, the system ensures better organization and management of resume data, making it easier for recruiters to compare candidates based on standardized parameters. It reduces human effort, minimizes bias, and increases transparency in the hiring process. The automation of resume screening also helps organizations reduce operational costs and improve overall efficiency.

Overall, the Resume Analyzer represents a modern approach to recruitment by integrating artificial intelligence techniques into traditional hiring systems. It not only improves the speed and accuracy of resume evaluation but also contributes to career development by guiding candidates toward skill enhancement and continuous learning. This makes it an essential tool in today's data-driven and competitive employment environment.

the system enhances decision-making by providing structured analytics and visual insights into candidate performance. It ensures that recruiters can easily compare multiple applicants based on standardized metrics, leading to more transparent and efficient hiring outcomes. This further strengthens the reliability and effectiveness of the recruitment process in organizations.

1.2 Problem Statement

Job searchers face a difficult barrier that might take two different shapes. To begin with, conventional resume modification processes find it difficult to satisfy the demands of different job applications. These techniques frequently lack precision and inventiveness. Because of the shortcomings of these traditional approaches, professionals are unable to adequately showcase their skills, which results in missed opportunities and a decreased likelihood of landing the job they want. The problem is made worse by the lack of a proactive tool that recommends certificates based on job opportunities. In the absence of strategic leadership, professionals are discouraged from making deliberate attempts to enhance their professional profiles and keep abreast of industry trends.

A creative resume analyzer that goes above and beyond conventional procedures is much needed to handle these two problems. This application requires the use of contemporary Natural Language Processing (NLP) techniques in order to assess job obligations and determine the best place to work. In addition to serving as an assessment tool, the Resume Analyzer ought to be a pro-active collaborator for experts, providing astute suggestions for certificates that correspond with the designated job description. Because it enables job searchers to proactively expand their skill sets and manage the intricacies of the modern labor market, this proactive component is essential.

The creation of a proactive and intelligent Resume Analyzer that integrates state-of-the-art natural language processing algorithms, predictive features, and customized certification suggestions is ultimately required to address this issue. In doing so, this instrument turns into a priceless tool that aids job seekers in effectively and mindfully navigating the complexities of the employment market.

Another significant problem is the lack of intelligent systems that guide candidates in improving their resumes and professional skills. Existing recruitment systems generally focus only on submission and screening, without providing feedback or suggestions for improvement. Candidates are often unaware of the specific skills, certifications, or qualifications required for different job roles. This creates a gap between industry expectations and candidate capabilities. Without proper guidance, job seekers struggle to enhance their profiles effectively.

Therefore, there is a strong need for an automated and intelligent Resume Analyzer system that

uses Natural Language Processing (NLP) techniques to evaluate resumes, extract key information, and match it with job requirements. Such a system should also provide personalized feedback, skill-gap analysis, and certification recommendations to help candidates improve their employability and career prospects.

1.3 Objectives

The main objectives of the proposed Resume Analyzer system are as follows:

- To implement Natural Language Processing (NLP) techniques for extracting important information such as personal details, skills, educational qualifications, and work experience from resumes.
- To develop an automated resume parsing system that accurately analyzes and organizes resume data in a structured format.
- To compare and evaluate multiple prompts in order to obtain optimal and accurate responses from the Large Language Model (LLM).
- To apply prompt engineering techniques for improving resume grading, feedback generation, and overall analysis performance.
- To provide candidates with instant feedback during the resume upload process for improving resume quality and job readiness.
- To develop a responsive and user-friendly interface that offers real-time resume scoring and skill-gap analysis.
- To assist recruiters in reducing manual effort and improving the efficiency of the recruitment process through intelligent automation.
- To improve the accuracy of candidate shortlisting by matching resume skills with job role expectations.
- To design a system capable of handling resumes in multiple formats such as PDF and DOCX.
- To integrate machine learning techniques for improving resume classification and recommendation accuracy.

1.4 Significance and Motivation of the Project Work

The Resume Analyzer project is notable since its objective is to improve the job application process. These days, a tool like this one is needed to accurately assess some of a candidate's skills because many employment marketplaces are very competitive. This project aims to assist people in properly showcasing themselves through polished narrative in order to improve their chances of landing desired positions.

The challenges people have when attempting to prepare resumes served as the inspiration for the notion. Many people find it challenging to identify critical abilities and align them with market demands. This is when the Resume Analyzer comes in handy. By providing tailored guidance and recommendations, it reduces this strain and increases the user's sense of comfort and empowerment during the job application process.

The second objective of the project is to advance an individual's professional growth. The Resume Analyzer helps people advance their careers and stay competitive by recommending relevant qualifications and adapting to changing market conditions. In the end, it'll be like having a really helpful professional success travel companion, with practical advice that directly affects job seekers' chances of getting hired.

1.5 Organization of Project Report

There are several chapters in this report. Chapter 1 gives a brief introduction of the Resume Analyzer using NLP techniques. Chapter 2 discusses the research work done to get an insight of the work that has been done in the field. Chapter 3 highlights the requirements and system development of the project. Chapter 4 gives the overview of the testing and test cases used during the making of the project. Chapter 5 presents the results of the project and its evaluation. Chapter 6 discusses the conclusion of the work done and the future scope of the project.

CHAPTER 2

LITERATURE SURVEY

2.1 Literature Survey

A literature survey is an important part of any research project because it in understanding previous work done in the same field. It provides knowledge about different techniques, methodologies, tools, advantages, and limitations used by researchers. In the field of automated resume screening and resume parsing, many researchers have used technologies like Natural Language Processing (NLP), Machine Learning, Deep Learning, Named Entity Recognition (NER), Optical Character Recognition (OCR), and Large Language Models (LLMs). The following literature survey discusses different research papers related to resume analysis and automated recruitment systems.

1. Gan, C., Zhang, Q., & Mori, T. (2024)

Gan, Zhang, and Mori proposed an advanced automated resume screening framework based on Large Language Model (LLM) agents. The study used Natural Language Processing, Open-source LLMs, Conditional Random Fields, and Vectorization Models. The main objective of this research was to improve the recruitment process by automating candidate selection and resume evaluation.

The methodology focused on using LLM agents for different stages of resume screening such as sentence classification, resume analysis, information extraction, and decision-making. The system was designed to reduce manual work and improve the efficiency of recruitment. The framework also improved communication between different stages of the resume screening process.

The results of the study showed that the use of LLM agents improved the effectiveness and performance of automated resume screening. The framework achieved better accuracy and faster processing compared to traditional systems. It also helped organizations in selecting suitable candidates more efficiently.

However, the study had several limitations. The framework faced risks related to data privacy and data compromise. The performance of the system depended heavily on model fine-tuning and human feedback. Another limitation was that different LLM models performed differently.

2. Jagwani, V., Meghani, S., Pai, K., & Dhage, S. (2023)

Jagwani and co-authors developed a resume analysis system using SpaCy and the Latent Dirichlet Allocation (LDA) model. The study mainly focused on extracting important information from resumes and categorizing resumes based on topics.

The methodology used SpaCy's Named Entity Recognition (NER) technique to identify important entities such as skills, education, experience, and technical knowledge from resumes. After extracting entities, the LDA model was used to assign topic probabilities and rank resumes according to relevance.

The proposed method successfully extracted relevant information from resumes and classified them into suitable categories. The system improved resume analysis by identifying important topics related to different job roles. It also reduced the manual effort required during candidate shortlisting.

Despite its advantages, the research had limitations. The system was not suitable for handling multiple career domains effectively. It lacked diversity in understanding resumes from different industries and job categories. Therefore, the model may not perform equally well for all types of resumes.

3. Bhor, S., Gupta, V., Nair, V., Shinde, H., & Kulkarni, M.S. (2021)

Bhor and co-authors proposed a resume parser system using Neural Networks, Conditional Random Fields (CRF), Convolutional Neural Networks (CNN), and Optical Character Recognition (OCR). The study aimed to automate the extraction and ranking of resume information.

The methodology used OCR and CNN for resume segmentation and extraction of textual information from resumes. Named Entity Recognition (NER) techniques were applied to understand domain-specific terms, technical skills, and keywords. The system extracted information such as personal details, educational qualifications, work experience, and technical skills.

The proposed model generated structured JSON outputs containing extracted resume information. It also ranked resumes according to company requirements and preferences. The system improved the efficiency of recruitment by reducing manual resume evaluation.

The limitations of this study were mainly related to NLP accuracy. The effectiveness of the parser depended on the quality of the resume and the performance of NLP techniques. Poorly formatted resumes could reduce the accuracy of information extraction. The model also required proper training for better performance.

4. Daryani, C., Chhabra, G.S., Patel, H., Chhabra, I.K., & Patel, R. (2020)

Daryani and co-authors developed an automated resume screening system using Natural Language Processing Toolkit, TF-IDF, and Cosine Similarity Matrix. The main objective of the study was to simplify the recruitment process for organizations.

The methodology involved extracting relevant information from resumes using NLP techniques. After preprocessing the text, TF-IDF was used to convert textual data into numerical vectors. Cosine similarity was then used to compare resumes with job descriptions and calculate similarity scores.

The proposed system helped organizations in automatically matching resumes with job requirements. It reduced the time required for manual screening and improved the recruitment process by selecting more relevant candidates.

However, the study had limitations. The system could not completely capture candidate qualifications accurately because NLP techniques may fail to understand context and hidden skills. The model also depended heavily on keywords, which sometimes affected the accuracy of candidate evaluation.

5. Green, N., Liu, M., & Murphy, D. (2020)

Green, Liu, and Murphy developed a resume analysis tool using NLTK, TF-IDF, and Cosine Similarity techniques. The aim of the study was to provide feedback and improvement suggestions for resumes.

The methodology used NLP techniques to analyze resumes and compare them with job descriptions. The system calculated similarity scores using cosine similarity and generated visual reports that highlighted strong and weak areas in resumes.

The results showed that the tool effectively analyzed resumes and provided useful suggestions for

Improvement. The visual reports helped candidates understand missing skills and improve resume quality according to job requirements.

The limitation of the study was that it did not discuss the biases associated with NLP techniques. NLP systems may unintentionally favor certain keywords or writing styles, which can affect fairness during resume evaluation.

6. Das, P., Pandey, M., & Rautaray, S.S. (2018)

Das and co-authors discussed the use of Big Data tools and R language for resume parsing and text analysis. The study mainly focused on handling large amounts of unstructured resume data.

The methodology included text preprocessing, tokenization, entity extraction, and data analysis using big data technologies. Different preprocessing techniques were applied to clean and organize resume data before extracting useful information.

The results showed that preprocessing methods effectively handled ambiguous and unstructured resume data. The research highlighted the importance of data cleaning and entity extraction in resume analysis systems.

However, the paper did not mention specific implementation challenges or limitations faced during the development process. Therefore, practical difficulties and system performance issues were not clearly discussed.

7. Sanyal, S., Hazra, S., Adhikary, S., & Ghosh, N. (2017)

Sanyal and co-authors proposed a resume parsing system using Natural Language Processing (NLP), NLTK Toolkit, and NoSQL Database. The main goal of the study was to extract resume information and store it efficiently for retrieval and analysis.

The methodology involved parsing resumes using NLP techniques and extracting entities such as personal details, education, skills, and work experience. The extracted information was stored in a NoSQL database for efficient management and retrieval.

The proposed method successfully extracted important entities from resumes and stored them in a structured database system. The use of NoSQL databases improved data storage flexibility and retrieval speed.

The limitation of the study was that the paper did not discuss any specific challenges or limitations faced during implementation. Issues related to scalability, accuracy, and resume formatting were not explained in detail.

Conclusion

The literature review highlights the evolution of resume screening methodologies, emphasizing the integration of Natural Language Processing (NLP) techniques for efficient candidate selection. While various models demonstrate successful extraction and analysis of resume data, limitations persist, including NLP bias, model accuracy, and domain-specific applicability. Future advancements should focus on mitigating biases, enhancing model accuracy, and ensuring adaptability across diverse industries. Overall, the reviewed studies provide valuable insights into the application of NLP in recruitment processes, paving the way for improved candidate evaluation and organizational efficiency.

2.2 Key Gaps in the Literature

The literature on automated resume screening and parsing has a few significant gaps and areas for possible improvement, based on the summaries that have been offered. The following are some observations:

A Basic Discussion of Ethical Issues: While Natural Language Processing (NLP) is mentioned by Chirag Daryania, Gurneet Singh Chhabra, Harsh Patel, Indrajeet Kaur Chhabra, Ruchi Patel [1], the ethical issues surrounding automated resume screening are not discussed. Crucial elements that must be considered and covered in the literature are ethical considerations, such as potential discrimination and bias in NLP models.

Insufficient Diversity in Career Fields: According to the second paper, it doesn't offer diversity across a range of career fields. In order to create inclusive hiring procedures, diversity is crucial. Scholars' ought to investigate strategies for augmenting the generalizability of their approaches across diverse businesses and occupational domains.

Lack of Restrictions Talk: Many publications don't specifically address the restrictions or difficulties encountered while putting their resume parsing algorithms into use. It is vital to acknowledge and deliberate upon the constraints in order to comprehend the pragmatic suitability and possible drawbacks of the suggested techniques.

Comparative analysis and evaluation metrics: While the articles describe the tools, approaches, and methodologies used, they do not go into great length about the assessment criteria that are used to gauge how well the suggested systems operate. A more comprehensive understanding of the efficacy of the suggested models may be obtained by conducting comparative evaluations with current practices or benchmarks.

NLP Technique Biases: Certain publications don't discuss the biases that come with using NLP approaches. Unfair results might arise from biases in language models, particularly in the recruitment process. An essential component of study in this field should be debating methods to reduce biases and enhance fairness.

Combining Big Data Methods and Tools: Papiya Das, Manjusha Pandey, Siddharth Swarup Rautaray[6] talk about using big data tools, but it doesn't go into great detail about how these tools work with the resume parsing paradigm. Researchers and practitioners may find great insights by comprehending the integration process and the advantages of using big data techniques.

CHAPTER 3

SYSTEM DEVELOPMENT

3.1 Requirements

In this stage we are going to list all the requirements that are needed in the model. The various Tools, Technologies and Libraries which are used.

Python: The Basis for Readability and All-Inclusive Libraries

Python is a popular, advanced, and versatile programming language. Its vast libraries and ease of reading have contributed to its global appeal. It is quite convenient for many operations like data analysis, web-building etc.

- **Readability:** The readability and relative ease at which to write and maintain coding for developers is highly emphasized in the syntax of Python, which plays an important role herein. This feature will be significant in large codebases and collaborative projects.
- **Large Library:** The enormous library and framework ecosystem within python is what makes it possible for the tool to fulfill multiple demands. These include TensorFlow, PyTorch, Django, Flask, NumPy, Pandas and many others. Development is made easier as these libraries are accessible while giving programmers strong instruments.

Streamlit: Making the Development of Web Applications Simpler

It is an open-source python package that was developed to simplify design and development of dashboards as well as web apps. It is good at developing application programs with focus on interaction and data presentation.

- **ase of Use:** Developers can design web applications with little difficulty thanks to Streamlit's user-friendly interface. It offers a straightforward script-based method that expedites the design and prototyping process.
- **Data Visualization:** Pre-built components available in the library can help enhance the development of dynamic and attractive visuals. Therefore, it becomes ideal for scenarios .

Text Analysis's Untapped Potential: The Natural Language Toolkit (NLTK)

A special Python package for use in NLP known as NLTK, which stands for Natural Language ToolKit. The program provides numerous functionalities, including sentiment analysis, stemming, and tokenization, to name some.

- **Text Analysis capabilities:** NLTK stands out as one of the top NLP tools because it has numerous capabilities for text analysis. For example, developers can employ features such as word tokenization, which breaks down text into words, stemming, reduces words back to their root form, among others.
- **Sentiment Analysis:** NLTK allows developers to determine whether a particular message is positive, negative, or neutral using sentiments analysis. It's especially essential when we talk about apps and the need of understanding user feedback.
- **Natural Language Processing:** Enhanced computational abilities of a computer using NLTK will be possible which makes it possible for programmers to create computers that can understand, read and produce text that sounds as if a human wrote it.

spaCy: Robust Advanced Natural Language Understanding

Another widely used tool in text analysis and natural language processing is the python module spaCy. This software is known for being effective, having pre-trained models, and supporting various tasks such as named entity recognition, tokenization, and part-of-speech tagging.

- **Efficiency and Speed:** SpaCy is commonly recognized to be fast in handling NLP tasks. It has been optimized for production use which will run large amount of text data using minimal resources.
- **Pre-Trained Models:** Developers can use pre-trained models for frequent tasks without requiring extensive training by applying the library's pre-trained models to a variety of NLP tasks. This guarantees consistent performance and shortens the development time.
- **Tokenization and Named Entity identification:** spaCy is a useful tool for applications requiring in-depth linguistic study since it is proficient in both named entity identification (identifying entities such as names, localities, etc.) and tokenization (splitting text into meaningful units).

PDFMiner: Merging Data from PDF Documents

A Python library called PDFMiner is used to extract text and structured data from PDF documents. Since many diverse sectors demand data extraction and analysis from PDF files, it is imperative to make this process easier.

- **Text Extraction:** Textual data can be handled and studied inside PDF files thanks to PDFMiner's ability to extract text content from PDF documents.
- **Structured Data Extraction:** PDFMiner can extract structured data from PDFs in addition to text. This is especially helpful for PDF documents that contain data in tabular or sorted formats.
- **Data analysis from PDFs:** Developers can extract insightful information from PDF documents by integrating PDF data into their analytical pipelines using PDFMiner.

PyResparser: Resume Parsing to Simplify Recruiting Procedures

A custom PyResparser module was created to parse and extract data from CVs and resumes. It is a useful tool for streamlining the hiring process and enhancing the effectiveness of candidate evaluation.

- **Resume Parsing:** PyResparser was developed specifically to parse resumes and retrieve vital data, including contacts, work history, education, and talents. The amount of manual labor required to review resumes is greatly decreased by this automation.
- **Structured Data Output:** Integrating the collected data into databases or application tracking systems is made easy by the library's structured data output. This guarantees unrestricted communication between parties during the recruiting process.
- **Recruitment efficacy:** PyResparser boosts recruitment efficacy by automating the preliminary stages of candidate evaluation. While the library compiles pertinent resume data, recruiters can concentrate on more complex job

Gemini API: LLM Model for better results

The Gemini API is a powerful platform made to enable traders and developers to programmatically include trading features and enable smooth connection with the Gemini cryptocurrency exchange.

- **Access Market Data:** Retrieve real-time market data for informed decision-making.

BeautifulSoup4: Web scrapping for better job matching

BeautifulSoup4 is a flexible Python module made to make online scraping easier and more accessible for developers to extract data from XML and HTML sources.

- **Parse HTML and XML:** Developers can parse web content and extract structured data using BeautifulSoup4's intuitive parsing methods, making it easy to navigate and extract information from complex document structures.
- **Navigate Document Structure:** The library provides tools for traversing the hierarchical structure of HTML and XML documents, enabling developers to locate and extract specific elements or data points efficiently.
- **Extract Information:** BeautifulSoup4 offers powerful methods for extracting information from web pages, including `find()`, `find_all()`, and `select()`, allowing developers to retrieve relevant data with ease.

Humantic API: Personality insights of the candidate according to resume

The Humantic API is designed to analyze text data using advanced artificial intelligence and natural language processing (NLP) techniques, with a specific focus on sentiment analysis and emotion recognition. Here's what the Humantic API does:

- **Sentiment Analysis:** The API evaluates text data to determine the sentiment expressed within it. It can classify text as positive, negative, or neutral based on the language used and the overall tone of the content.
- **Emotion Recognition:** In addition to sentiment analysis, the Humantic API can recognize specific emotions conveyed in the text. It can identify emotions such as happiness, sadness, anger, fear, and others, providing insights into the emotional context of the text.
- **Customizable Parameters:** The API offers customizable parameters that allow users to fine-tune the analysis according to their specific requirements. Users can adjust parameters such as language settings, sentiment thresholds, and emotion categories to optimize results for different use cases and applications.

3.1 Project Design and Architecture

A major development in resume analysis and optimisation is represented by the Resume Analysis Application using Latent Language Models (LLM). The software provides job searchers wishing to improve their resumes and increase their chances of success in the cutthroat job market with a complete solution that combines advanced natural language processing techniques with an intuitive UI.

One of the main advantages of the programme is its intuitive and smooth user interface, which has been meticulously created. Users are led through the resume analysis process by an aesthetically pleasing and tidy interface as soon as they launch the programme. Resumé uploading in PDF, DOC, and DOCX formats is made easy for users by the interface's straightforward drag-and-drop file upload capability. All along the way, users are helped and made to comprehend every stage of the analysis process by the clear explanations and tooltips.

Following an upload of a resume, the programme starts the preparation phase, which involves normalising and cleaning the text to guarantee correctness and consistency. Ensuring uniform resume content and analysis readiness depends on this preprocessing stage. To make more analysis easier, formatting errors and irrelevant material are eliminated, and the text is standardised in design.

Modern natural language processing models trained on enormous volumes of text data, known as BERT or GPT, provide the basis of the analysis engine of the programme. These models let the programme to decipher the semantic background of resume text and get useful information from it. Employing sophisticated methods such entity identification, semantic similarity, and context-aware language understanding, the software can extract important data from the resume including work experience, education, abilities, certifications, and accomplishments.

The application's analysis goes beyond simple keyword matching to give customers thorough understanding of the advantages, disadvantages, and areas for development of their resume. Clients get customised suggestions based on their particular skill set, professional objectives, and market need. Among these suggestions can be ways to highlight important qualifications and experiences, streamline resume material, and make layout changes. To help users fill up skill gaps and progress in their professions, the software also compares extracted abilities and experiences with job criteria

Industry trends and offers suggestions for pertinent courses, certifications, and training programmes.

The analytical algorithms and recommendations are always being updated as people use the programme and offer input. Through this iterative approach, users may optimise their resumes and succeed in their job hunt by knowing that the application is always current and responsive to changing user needs and market trends. All things considered, the Resume Analysis Application with Latent Language Models (LLM) is a major development in resume analysis technology that gives job searchers practical information and customised suggestions for professional growth.

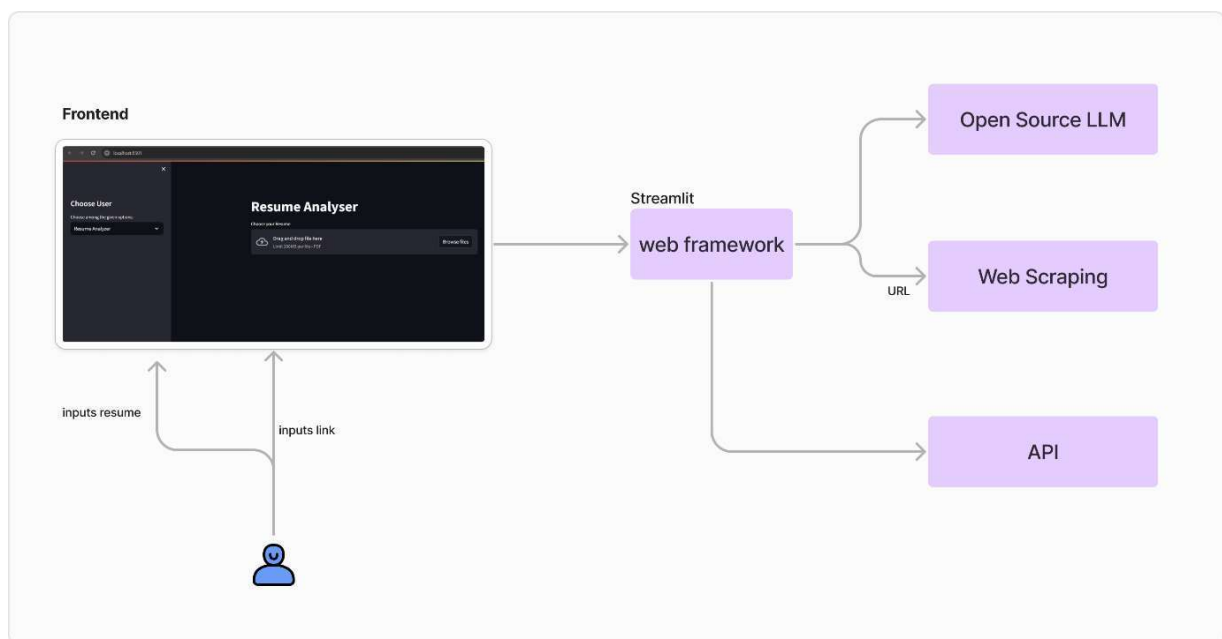


Fig.1: Flow of the app

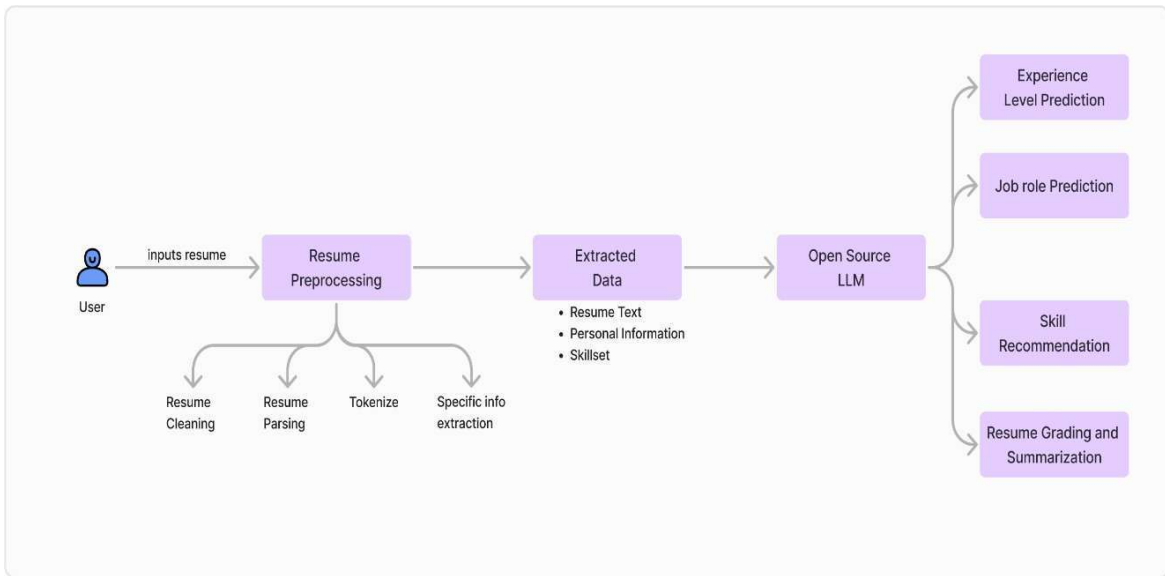


Fig.2 Flow of Resume Analyzer

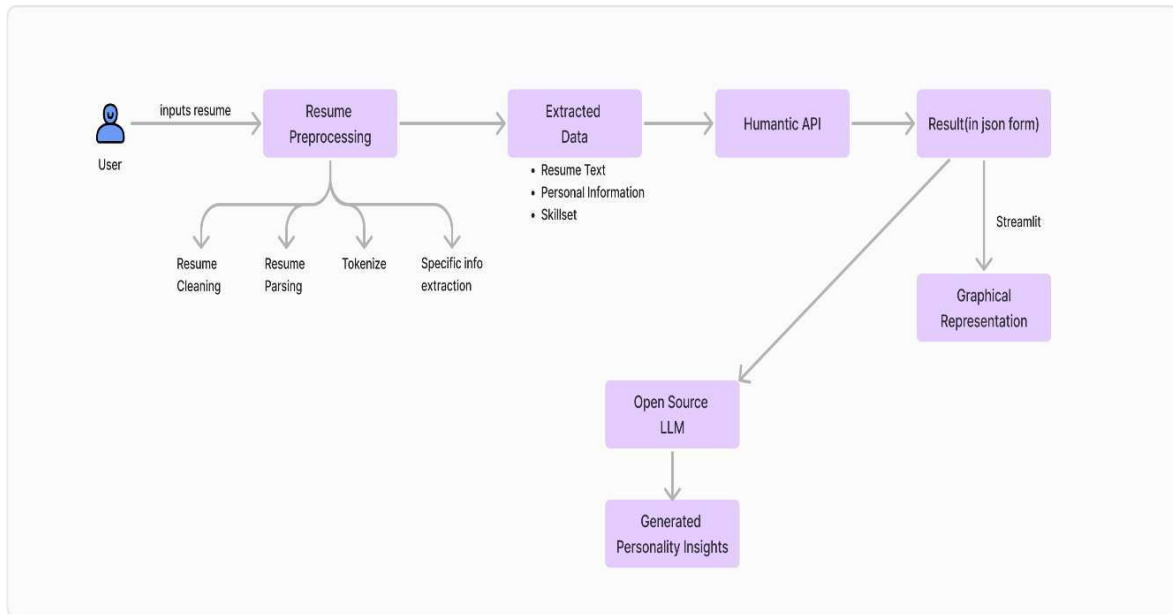


Fig.3 Flow of Personality Insights

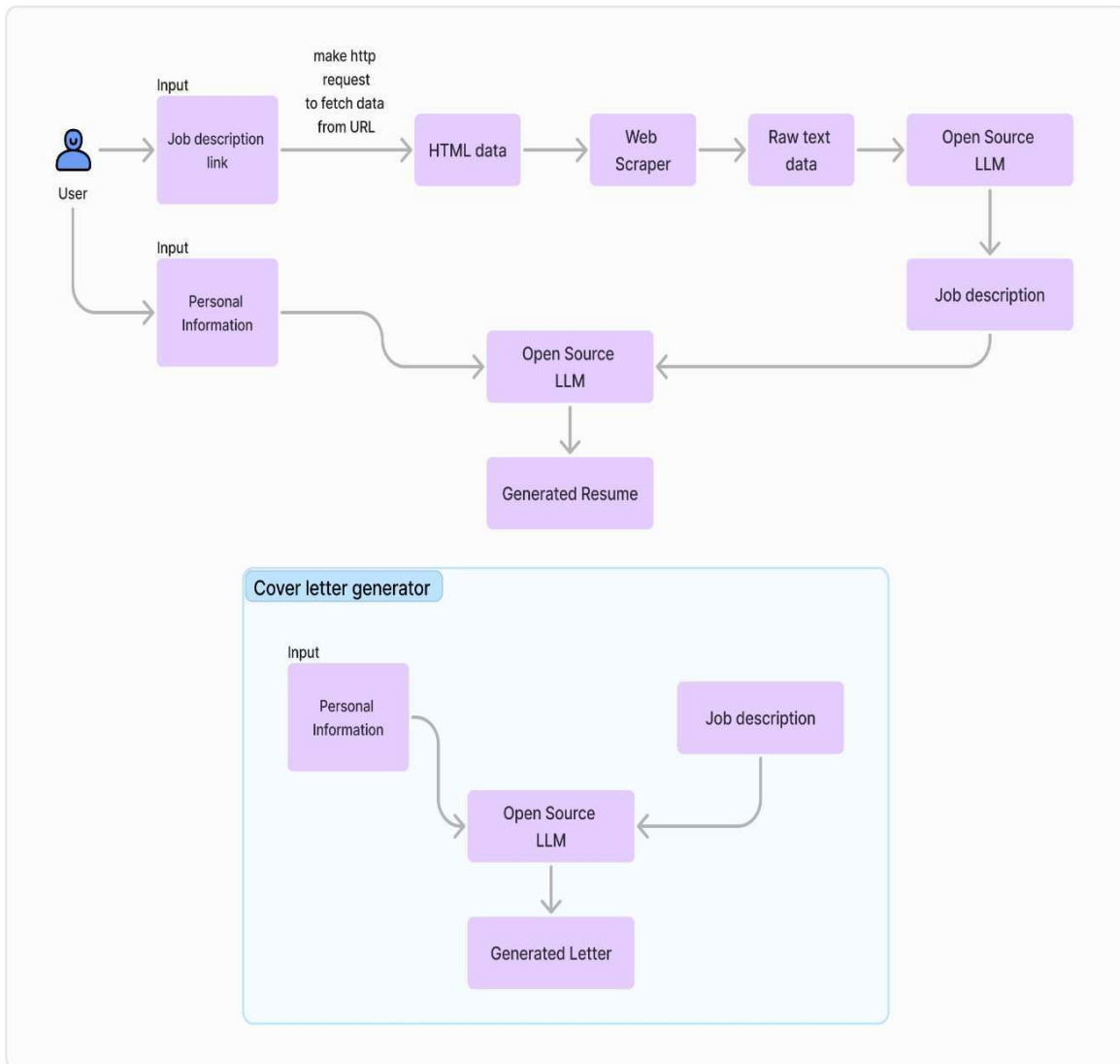


Fig.4 Flow of Resume Builder

3.2 Data Preparation

Automated systems are now essential for quickly processing and evaluating candidate data in the modern job market, when hiring managers and recruiters are deluged with resumes. The resume cleansing and resume parsing are two essential parts of these systems. We'll examine the nuances of both processes in this thorough overview, looking at their goals, important duties, underlying methods, and overall integration inside automated systems.

Resume Cleaning:

The first stage of the automated resume analysis pipeline is resume cleaning, which deals with the variety of formats, structures, and quality variances that are present in resumes. Preprocessing the retrieved data, standardizing the information, and removing inconsistencies are the main goals in order to produce a uniform and clean dataset for further analysis. This is crucial to guaranteeing the automated resume analysis system's efficacy and correctness.

Important Assignments:

1. **Standardization of Formatting:** Resume formats, such as font styles, sizes, and document layouts, are frequently varied. Standardizing these components enables the cleaning procedure to produce a visually uniform and consistent dataset. This entails establishing a uniform structure for all resumes and standardizing date formats and font styles.
2. **Noise Removal:** - Resumes may include extraneous information, graphics, headers, or footers that aren't necessary. Finding and eliminating this kind of noise allows the dataset to be reduced to its most basic parts. This is a crucial step in minimizing irrelevant data that can impede precise analysis.
3. **Typographical Error Correction:** - Inaccuracies in names, addresses, and other pertinent information might make analysis difficult. During the cleaning process, algorithms are used to find and fix typographical problems, guaranteeing that the collected data is error-free.

The process of eliminating duplicate material from resumes involves identifying and removing instances where specific data or phrases are used more than once. To make the dataset more streamlined, the cleaning process entails locating and eliminating these redundancies. This improves productivity and guards against errors resulting from redundancy in later analysis.

Approaches: - Regular expressions for pattern matching, string matching algorithms for phrase recognition, and context-aware noise reduction using natural language processing (NLP) approaches are some of the techniques used during resume cleaning. Moreover, machine learning models can be applied to more difficult jobs like standardizing formatting.

The results of the resume cleaning procedure serve as the basis for the automated system's later phases. The information input into the resume processing step is consistent and free of extraneous noise when the data is cleaned and standardized, which creates the conditions for the precise extraction of crucial information.

Resume Parsing:

The next step in automated resume analysis is called resume parsing, and it involves methodically organizing and structuring the cleaned and standardized data for simple analysis and retrieval. The main goal is to gather precise information in an organized manner regarding a candidate's experiences, abilities, and credentials. During the hiring process, insights are generated, and judgments are made based on this structured data.

1. **Entity Extraction:** - Extracting important entities from the cleansed data is one of the main jobs in resume parsing. This entails locating and retrieving personal data (name, contact information), as well as data on educational background, professional experience, certificates, and other pertinent information. For this objective, Named Entity Recognition (NER) models are frequently used.

2. **Contextual Understanding:** - To comprehend the contextual relationships among various entities in the resume, algorithms are used. For example, contextual comprehension is necessary to discern between a common term and a specific ability. This guarantees that the system understands the text's complex meaning correctly.

3. **The Hierarchical Structure:** A hierarchical structure reflecting the innate relationships between various things is created from the retrieved data. This organization makes it easier to get particular details for later analysis. For example, the hierarchical structure can group abilities, employment history, and educational background into different branches for convenient access.

Job Titles, and making sure date formats are consistent. Normalization is essential to removing discrepancies that could occur from variations in the information that candidates choose to include on their resumes.

4. **Database Integration:** - For archiving and retrieval, the parsed data is incorporated into a database. This database facilitates effective searching and analysis by acting as a central store for candidate data. The system's scalability and accessibility are improved by database integration.

Techniques: - During resume parsing, entity extraction is frequently accomplished using Named Entity Recognition (NER) models, rule-based algorithms, and machine learning techniques. Contextual comprehension is enhanced by methods like semantic analysis and pattern matching. Database management systems are used to store and arrange parsed data efficiently.

A structured and ordered dataset is produced as a result of the resume parsing process, which is very helpful for further analysis. Because the structured data is kept in a database, it can be quickly retrieved for searching, creating insights, and helping hiring managers make well-informed decisions.

3.3 Implementation

Setup the environment: Create a python virtual environment and install the required libraries and dependencies.

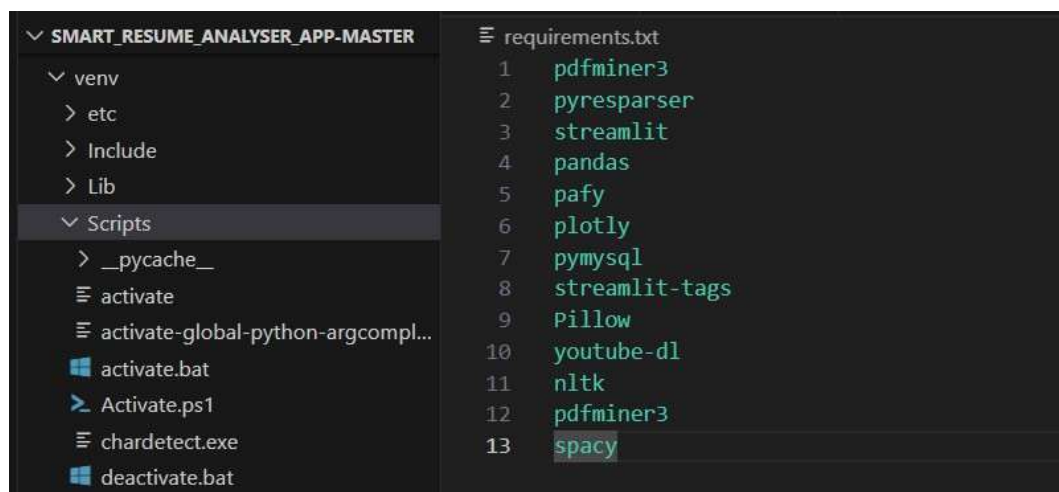


Fig. 6: Environment setup

Data Preparation: Load and preprocess the resume data.

```
def pdf_reader(file):
    resource_manager = PDFResourceManager()
    fake_file_handle = io.StringIO()
    converter = TextConverter(resource_manager, fake_file_handle, laparams=LAParams())
    page_interpreter = PDFPageInterpreter(resource_manager, converter)
    with open(file, 'rb') as fh:
        for page in PDFPage.get_pages(fh,
                                      caching=True,
                                      check_extractable=True):
            page_interpreter.process_page(page)
            print(page)
        text = fake_file_handle.getvalue()

    # close open handles
    converter.close()
    fake_file_handle.close()
    return text

def show_pdf(file_path):
    with open(file_path, "rb") as f:
        base64_pdf = base64.b64encode(f.read()).decode('utf-8')
    # pdf_display = f'<embed src="data:application/pdf;base64,{base64_pdf}" width="700" height="1000" type="application/pdf">'
    pdf_display = f'<iframe src="data:application/pdf;base64,{base64_pdf}" width="700" height="1000" type="application/pdf"></iframe>'
    st.markdown(pdf_display, unsafe_allow_html=True)
```

Fig. 7: Data preparation

Instruction Engineering: Prompt for the model to execute the task

```
def get_response(resume_text):
    response = palm.generate_text(prompt=f"Grade this resume: {resume_text} on a scale of 1-10. Grade it on the basis of education")
    # print(response.result)
    return response.result

def get_role(resume_text):
    response = palm.generate_text(prompt=f"Give answer in one word, according to the resume: {resume_text}, candidate has experience in which role?"
    # print(response.result)
    return response.result

def get_level(resume_text):
    response = palm.generate_text(prompt=f"Give answer in one word, according to the resume: {resume_text}, what is the level of experience?"
    # print(response.result)
    return response.result

def get_skills(resume_text):
    response = palm.generate_text(prompt=f"Give answer in form of string separated by commas, the candidate has following skills: {resume_text}")
    # print(response.result)
    return response.result
```

Fig. 8: Prompts for LLM to execute

Resume Builder: Helps in generating resume for a special job posting

```
response = requests.get(job_link, headers={'User-Agent': random.choice(user_agents)})
print(response.status_code)
content = response.text

soup = BeautifulSoup(content, "html.parser")
job_description_raw_text = soup.get_text().replace('\n', '').strip()
print(job_description_raw_text)

# client = Client()
You, 1 second ago • Uncommitted changes
job_description = palm.generate_text(prompt=f"Get the job decription from the following text: {job_description_raw_text}")
print(job_description.result)
resume_output = palm.generate_text(prompt=f"Get the job decription from the following text: {job_description_raw_text}")
# print(resume_output.result)
st.success(resume_output.result)
result = st.button("Cover letter Generator")
if result:
    cv_output = palm.generate_text(prompt=f"Write a cover letter for the specific job based on the following job desc")
    st.success(cv_output.result)
```

Fig. 9: Resume Builder execution

Resume Matcher: Matching the resume with the job posting to find the perfect find according to the resume

```
response = requests.get(job_link, headers={'User-Agent': random.choice(user_agents)})
print(response.status_code)
content = response.text

soup = BeautifulSoup(content, "html.parser")
job_description_raw_text = soup.get_text().replace('\n', '').strip()
print(job_description_raw_text)

prompt = ""
You are a job role description extractor. You will take the text and extract the job role description
model = palm.GenerativeModel('gemini-pro')
response = model.generate_content(prompt + job_description_raw_text)
print(response.text)

# job_description = palm.generate_text(prompt=f"Return the job decription from the given text: {job_description_r")

resume_output = palm.generate_text(prompt=f"Check if the candidate with the resume : {resume_text} is a good match")
# print(resume_output.result)
st.success(resume_output.result)
if "Yes" in resume_output.result:
    result = st.button("Cover letter generator")
    if result:
        cv_output = palm.generate_text(prompt=f"Write a Cover letter for the specific job based on the following")
        st.success(cv_output.result)
```

Fig. 10: Resume Matcher implementation

```
if "Yes" in resume_output.result:
    result = st.button("Cover letter generator")
    if result:
        cv_output = palm.generate_text(prompt=f"Write a Cover letter for the specific job based on the following")
        st.success(cv_output.result)
```

Fig 11: Cover letter generation

```

else:
    pdf_file = st.file_uploader("Choose your Resume", type=["pdf"])
    if pdf_file is not None:
        resume_data, save_image_path = file_uploader(pdf_file)
        if resume_data:
            ## Get the whole resume data
            resume_text = pdf_reader(save_image_path)
            BASE_URL = "https://api.humantic.ai/v1/user-profile/create" # Base URL for create endpoint
            headers = {
                'Content-Type': 'application/json'
            }
            API_KEY = os.getenv("HUMANTIC_API_KEY")
            USER_ID = "201132@gmail.com" # or, any unique identifier

            url = f"{BASE_URL}?apikey={API_KEY}&userid={USER_ID}&analysistype=talent"

            data = resume_text
            payload = json.dumps(data)

            response = requests.request("POST", url, data=payload, headers=headers)
            print(response.status_code, response.text)
            time.sleep(7)
            FETCH_URL = "https://api.humantic.ai/v1/user-profile" # Base URL for the FETCH endpoint

            url = f"{FETCH_URL}?apikey={API_KEY}&id={USER_ID}"

```

Fig. 12: Personality Insights generator

Integration: Combine the NLP model, job prediction, and certification recommendation modules into a cohesive system by creating a streamlit application.

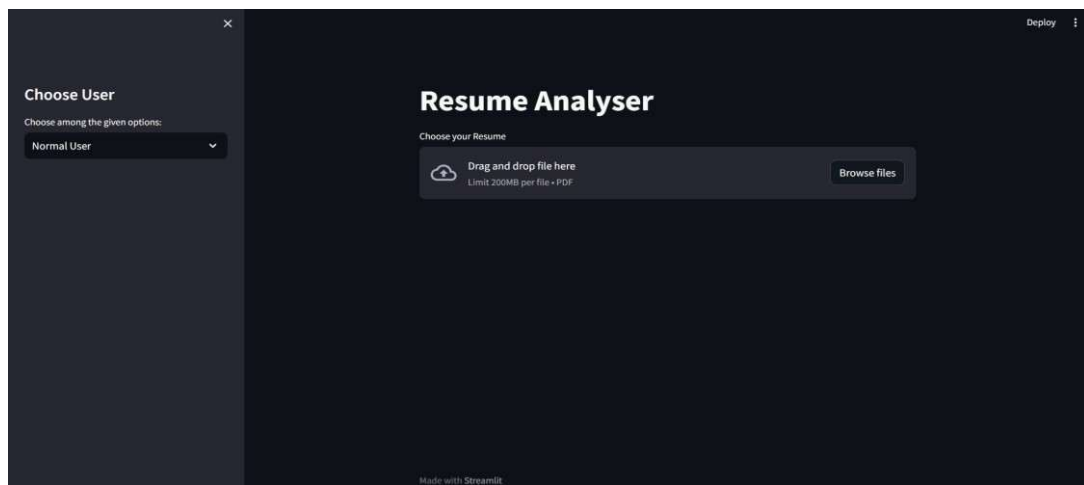


Fig 13: User Interface

CHAPTER 4

CORE FEATURES & MODULES

4.1 Resume Upload & Ingestion Module

The ingestion module provides a seamless multi-channel entry point for resume submission. Users can upload resumes individually, in batch (up to 500 at once), or via LinkedIn profile import. The module supports all common document formats and performs immediate validation feedback.

Supported Formats & Validation

- PDF (including scanned documents with OCR fallback)
- Microsoft Word (.docx, .doc)
- Plain Text (.txt, .rtf)
- OpenDocument Format (.odt)
- HTML resume format

HTML resume format

Ingestion Pipeline Steps

1. File format validation and malware scanning
2. Metadata extraction (file size, creation date, author)
3. Character encoding detection and normalization
4. Layout analysis (single-column, multi-column, tabular)
5. Text layer extraction with coordinate preservation
6. OCR processing for image-based or scanned documents
7. Document fingerprinting for deduplication

4.2 Intelligent Parsing Module

The parsing module transforms raw resume text into a structured, machine-readable representation. It identifies and extracts over 40 distinct data fields organized into logical sections.

Section	Extracted Fields
Personal Info	Full name, email, phone, location, LinkedIn, GitHub, portfolio URL
Work Experience	Company, title, start/end date, duration, responsibilities, achievements, tech stack used
Education	Institution, degree, field of study, GPA, graduation year, honors, relevant coursework
Skills	Technical skills, soft skills, proficiency levels, years of experience per skill
Certifications	Certification name, issuing body, issue date, expiry date, credential ID
Projects	Project name, description, technologies, role, GitHub link, impact metrics
Publications	Title, venue, authors, date, DOI, abstract

4.3 Scoring Engine Module

The scoring engine is the analytical heart of the system. It evaluates resumes against job descriptions across six weighted dimensions, producing an overall compatibility score from 0 to 100.

Scoring Dimension	Weight	Description
Technical Skills Match	35%	Direct skill overlap using semantic matching and taxonomy
Experience Relevance	25%	Role similarity, industry match, seniority alignment
Educational Fit	15%	Degree relevance, institution quality, field alignment
Keyword & JD Alignment	12%	Coverage of required and preferred JD terms
Soft Skills & Culture	8%	Leadership, communication, teamwork indicators
Achievements & Impact	5%	Quantified accomplishments, awards, recognitions

4.4 Recommendation Engine

After scoring, the recommendation engine generates a personalized improvement plan for each candidate. Recommendations are ranked by impact potential and categorized by effort required.

- High-Impact, Low-Effort: Add specific missing keywords; mention certifications already held
- High-Impact, High-Effort: Acquire recommended certifications; gain experience in gap areas
- Medium-Impact: Quantify existing achievements; improve formatting and ATS readability
- Best Practices: Grammar and clarity improvements; professional summary optimization

4.5 Analytics Dashboard

The recruiter dashboard provides aggregate insights across all analyzed resumes for a given job posting. Key dashboard components include:

- Score distribution histogram and percentile rankings
- Skill gap heatmap across candidate pool
- Top candidate shortlist with side-by-side comparison
- Diversity metrics and demographic distribution (anonymized)
- Pipeline funnel analytics (applied, shortlisted, interviewed, offered)
- Historical trend analysis and benchmarking across previous hires
- Resume screening performance reports
- Job-role wise candidate analysis
- Real-time recruitment progress tracking
- Candidate experience and qualification comparison
- AI-based hiring recommendations
- Department-wise recruitment statistics
- Skill matching percentage analysis
- Interactive charts and graphical reports
- Automated report generation and export facility
- Recruiter activity monitoring and insights

CHAPTER 5

USER INTERFACE & EXPERIENCE

5.1 Design Philosophy

The user interface is designed around three core principles: clarity (complex AI outputs presented in human-understandable formats), efficiency (critical actions reachable in 2 clicks or fewer), and trust (transparent explanations for all AI-generated scores and recommendations). The design system follows Material Design 3 guidelines with custom branding.

5.2 Recruiter Interface

5.2.1 Dashboard View

The recruiter dashboard provides an at-a-glance overview of all active job requisitions, their application volumes, average candidate scores, and pipeline status. Key widgets include:

- Active Jobs Summary: Card grid showing open positions with candidate count and average score
- Top Candidates Feed: Rolling list of recently analyzed high-scoring applicants
- Score Distribution Chart: Histogram of candidate scores per selected job posting
- Activity Timeline: Recent analysis completions and team actions
- Quick Actions: Upload resume, create new JD, start batch analysis

5.2.2 Candidate Comparison View

Side-by-side comparison of up to 4 candidates against a job description, showing normalized score breakdowns, skill match visualizations, and key differentiators. Radar charts display multi-dimensional fit profiles, making complex score data immediately comprehensible.

5.3 Candidate Interface

5.3.1 Resume Analysis View

The candidate-facing interface provides a comprehensive analysis report in an accessible, motivating format. The report is structured to educate candidates on what matters to employers while maintaining a constructive and encouraging tone.

- Overall Score Display: Large circular progress indicator with score and percentile
- Dimension Breakdown: Horizontal bar charts for each scoring dimension with explanations
- Skill Match Matrix: Color-coded grid showing matched, missing, and bonus skills
- Improvement Roadmap: Prioritized action items with estimated impact per recommendation
- Resume Preview: Annotated view highlighting strengths and areas for improvement

5.4 Accessibility Standards

The application meets WCAG 2.1 AA accessibility standards, ensuring usability for all users including those with visual, motor, or cognitive disabilities.

- Screen reader compatibility (tested with NVDA, JAWS,)
- Keyboard-only navigation for all core workflows
- Color contrast ratios meeting AA requirements (4.5:1 for text, 3:1 for UI components)
- Resizable text up to 200% without loss of content or functionality
- Alternative text for all informational images and icons

5.5 Responsive Design

Breakpoint	Target Device	Layout Adjustments
xs: < 576px	Mobile phones	Single column, bottom navigation, condensed charts
sm: 576-768px	Large phones, small tablets	Two-column cards, side-scrollable tables
md: 768-992px	Tablets, small laptops	Collapsible sidebar, full chart views
lg: > 992px	Desktops, large monitors	Full layout with persistent sidebar

5.6 Key Challenges

Creative Resumes and Unstructured Data: For automated systems, managing creative resume forms and unstructured data presents a major difficulty. Certain candidates use eye-catching but non-traditional layouts, like infographics, graphics, or nonlinear structures. It takes sophisticated natural language processing (NLP) methods and machine learning models that can accurately adjust to a variety of resume formats in order to extract pertinent information from such resumes. Finding the right balance in data extraction between precision and flexibility is still a difficult task.

Ambiguities and Nuances in Language: Resume analysis systems are constantly faced with the subtleties and ambiguities found in natural language. Frequently, candidates utilize terminologies that are context-dependent, industry-specific, or abbreviated in ways that differ throughout industries. Comprehending the minute variations in linguistic subtleties is essential to precise entity identification and contextual comprehension. To improve the system's language understanding capabilities, this task calls for constant training on a variety of datasets, semantic analysis, and NLP algorithm improvement.

Algorithmic Decision-Making: Fairness and Bias: Automated resume analysis tools have the potential to unintentionally reinforce prior data biases, producing unfair or discriminating results. Biases can appear at several phases, from the extraction of data to the course recommendation based on talents detected. The introduction of ethical principles, continuous monitoring, and thorough testing are necessary to guarantee fairness in algorithmic decision-making. To reduce bias and advance fairness in the hiring process, tactics including debiasing algorithms, broadening training datasets, and frequent audits are crucial.

Changing Industry Demands and Skill Sets: Resume analysis systems have challenges due to the dynamic nature of skill requirements and the quick evolution of industries. The changing nature of work roles and the emergence of new technology may make traditional keyword-based techniques obsolete. Systems must be flexible, constantly upgrading their knowledge of vocabulary and expertise unique to a certain business. It becomes essential to integrate with real-time job market data and industry professional feedback methods to stay up to date with evolving trends and guarantee that the system's recommendations are still applicable.

Data security and privacy concerns: Strong data security measures are necessary since resumes contain sensitive personal information, which causes privacy concerns.

Resume Formats Are Not Standardized: The diversity of resume forms and structures makes standardization difficult. Different titles, templates, and information structures might make it difficult to extract and parse data accurately. It is difficult to establish a standard resume format that is approved by all employers because resumes are customized. Accuracy and consistency must be preserved in automated systems when they are adjusted to various structures. The creation of sophisticated parsing algorithms that can see and classifying data even in the lack of a standard format is necessary to find solutions.

Integrating with People Who Make Decisions: Achieving synergy between technology and human expertise is a difficulty when it comes to effective integration between automated systems and human decision-makers. Improving the automated analytical process's ability to make decisions requires an understanding of and integration with human insights, preferences, and subjective assessments. Creating interfaces that promote teamwork and utilizing user input to continuously enhance the system are key components in striking a balance between automated efficiency and human intuition.

CHAPTER 6 TESTING

6.1 Testing Strategy

Unit Tests: The NLP Model Unit Test, which verifies the precision and efficacy of the Natural Language Processing (NLP) model, is a crucial component of the Resume Analyzer project's overall testing approach. This test seeks to ascertain whether the NLP model accurately extracts pertinent data from a range of resumes. In this context, it entails presenting the model with many templates that depict diverse conventions, styles or forms adopted by industries. In the unit test, the model determines if it is able to process input items in text format correctly, detect key elements as well as generate useful information about work experience, academic qualification, and other related details. Throughput assessment on such a microscopic level makes sure that the interpretation of the NLP model matches the complex subtleties of actual practice. The successful execution of the NLP Model Unit Test demonstrates the validity of the model and acts as the basis for the competence of the Resume Analyzer to analyze different resumes correctly.

Integration Tests: To test the reliability of the Resume Analyzer system, the end-to-end testing should be carried out to ensure that interfaces match perfectly. Integration is necessary for the NLP model, certification recommendation, and job prediction modules to function as a cohesive unit. This entails ensuring that data flows seamlessly between modules and that consistency and correctness are always upheld. Next, end-to-end testing is used to evaluate the system's functionality in detail. This testing begins with the resume input and concludes with the outputs of job prediction and certification recommendation. This test simulates real-world environments to confirm that the related parts work well together and produce correct results. A well-functioning system that assesses resumes, predicts job openings, and integrated recommends relevant certificates is one that integrates and tests well from start to finish. Regular validation and refinement of these procedures improves the Resume Analyzer project's overall dependability and efficacy.

Hardcoded Data Validation: Based on the job prediction and the predefined hardcoded data, this crucial Resume Analyzer project component verifies that the certifications that are suggested are valid and pertinent. This validation process includes a careful comparison of the anticipated employment roles with the relevant credentials in the hardcoded data. The system does this in an

Attempt to confirm that the recommended certifications satisfy the standards and industry norms pertaining to the expected work positions. In addition to increasing the accuracy of the certification recommendations, this thorough verification procedure guards against mistakes that could arise from discrepancies between the hardcoded data and the dynamic nature of labor markets. The automated resume analysis process is made more reliable and effective overall by the Resume Analyzer system's ongoing validation and updating of its capacity to offer pertinent and helpful certification recommendations.

User-Interface Tests: The success of the project depends on efficient input validation, which ensures that the Resume Analyzer system handles a range of resume formats and input styles with grace. This means setting up dependable procedures to interpret and preprocess different resume formats while considering variations in document types, layouts, and content configurations. Its capacity to manage this variability makes the system more flexible and easier to use, allowing it to easily accept the diverse range of resumes that are submitted for review. But output validation is equally crucial because it verifies that the system displays certification and job projections accurately. This validation process contributes to the development of trust in the system's analytical capabilities by guaranteeing that end users receive reliable and accurate information. With its emphasis on thorough input and output validation, the Resume Analyzer distinguishes itself as a reliable tool that assists users in customizing their resumes for specific job vacancies. This ensures excellent user satisfaction.

Tests of Robustness: The robustness of the Resume Analyzer is assessed by examining how effectively it manages resumes containing ambiguous or imprecise information and by verifying the strength of its error-handling techniques. The system should be able to analyze resumes with unique forms or vague job descriptions with ease, displaying a flexible approach to language use. In addition, a thorough evaluation of error handling is required to ensure that users receive helpful error messages that aid in issue solving and that the system adjusts seamlessly to unanticipated events. This dual evaluation emphasizes the importance of the system's versatility and user-friendliness. The system's ability to navigate through intricate resume information and accurately highlight any issues improves the Resume Analyzer project's overall dependability and usability.

Cross-platform and cross-browser tests: These are critical to confirming the system compatibility of the Resume Analyzer. To ensure that the system functions consistently and faultlessly across a range of devices and web browsers, extensive testing is required. The project

AIMS to confirm that users can consistently access and use the Resume Analyzer regardless of their preferred browser (Chrome, Firefox, Safari, or Edge) or operating system (Windows, macOS, or Linux) through extensive compatibility testing. This comprehensive assessment guarantees that the application will be viewed by a larger audience and enhances the user experience, which emphasizes the system's accessibility and adaptability even more. Regardless of the technical environment in which it operates, the Resume Analyzer project places a high priority on cross-browser and cross-platform compatibility to deliver a dependable and consistent user experience.

6.2 Test Cases and Outcomes

In this section test cases are used to assess the system and the anticipated results are covered in this section.

Test Case no. 1

Test Case No.	1
Objective:	To test whether the user is able to upload file from the file explorer.

Test Data	Expected Result	Actual Result
Click Upload Button and upload file	Open file explorer and upload file.	Open file explorer and upload file.

Test Result	The above result indicates that the admin is able click upload button and upload files.
-------------	---

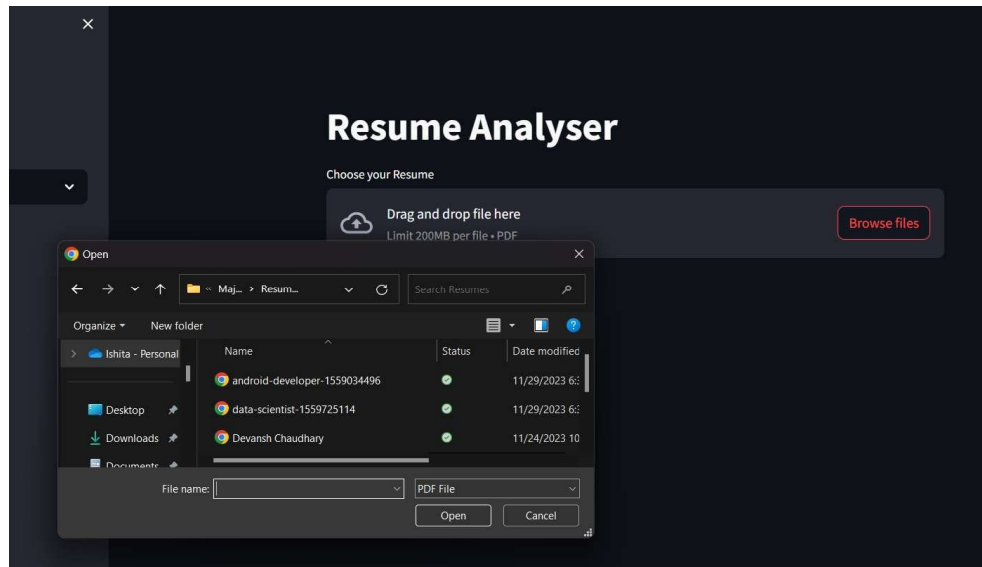


Fig. 14: Result of Test Case no.1

Test Case no. 2

Test Case No.	2
Objective:	To test whether the necessary information from the resume is parsed.

Test Data	Expected Result	Actual Result
Upload the resume	Show name, email, mobile number, technical skills, soft skills.	Show name, email, mobile number, technical skills, soft skills.

Test Result	The above result indicates that the necessary information from the resume has been parsed.
-------------	--

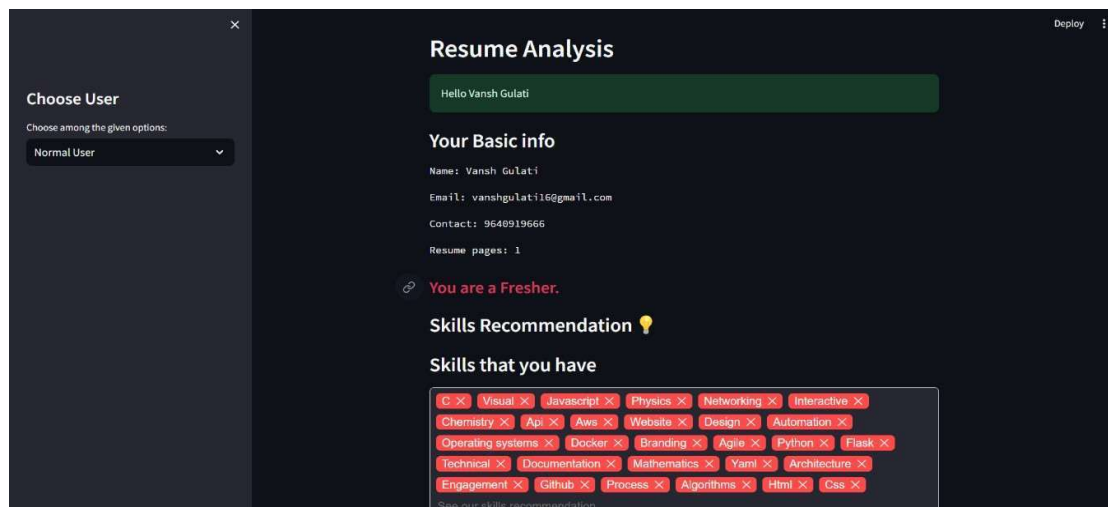


Fig. 15: Result of Test Case no. 2

Test Case no. 3

Test Case No.	3
Objective:	To test whether the job description is being scraped or not.

Test Data	Expected Result	Actual Result
Input the link	Shows job description.	Shows job description.

Test Result	The above result indicates that the job description is scraped.
-------------	---

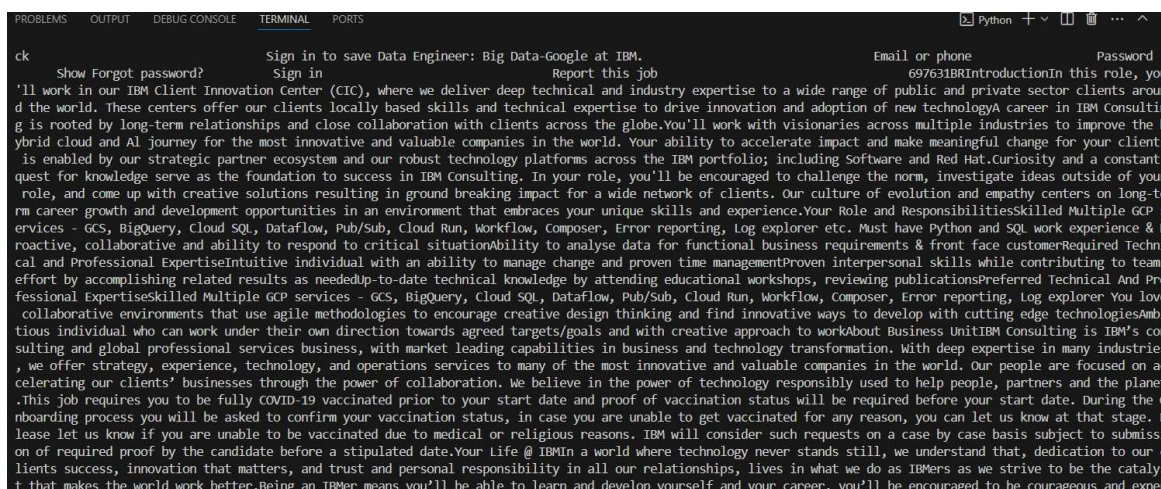


Fig. 16: Result Test Case no. 3

6.1 TESTING STRATEGY & RESULTS

6.1.1 Testing Philosophy

The project follows a test-driven development (TDD) approach with a comprehensive testing pyramid: a large base of unit tests, a smaller set of integration tests, and a focused suite of end-to-end tests. AI/ML components require specialized testing strategies beyond standard software testing practices.

6.2.2 Testing Types

6.2.1 Unit Testing

Component	Framework	Coverage	Tests
Parsing Engine	pytest	94%	847
Scoring Engine	pytest	91%	623
API Layer	pytest + httpx	96%	412
Frontend Components	Jest + RTL	88%	534

6.2.2 Integration Testing

Integration tests verify correct interaction between microservices, database operations, message queue workflows, and third-party integrations. Tests run against a containerized test environment using Docker Compose, mirroring the production configuration.

- Service-to-service communication tests (gRPC and REST)
- Database transaction and rollback tests under concurrent load
- Kafka message publishing and consumption tests with exactly-once semantics
- Third-party API integration tests using recorded fixtures (VCR)

6.2.3 ML Model Testing

- Statistical tests verifying model output distributions match expected ranges
- Invariance tests: score should not change if irrelevant information changes
- Monotonicity tests: adding a required skill should never decrease the score
- Fairness/bias tests: score differences across demographic groups within tolerance

6.2.4 Performance Testing

Test Type	Tool	Target	Result
Load Testing	Locust	500 concurrent users	PASS - p99 < 2s
Stress Testing	Locust	2,000 concurrent users	PASS - degraded, no crash
Soak Testing	Locust	100 users for 24 hours	PASS - no memory leaks
Spike Testing	Locust	10x normal load in 30s	PASS - auto-scaled in 45s

6.2.5 Security Testing

- OWASP ZAP automated vulnerability scanning in CI/CD pipeline
- Manual penetration testing conducted by external security firm (quarterly)
- Dependency vulnerability scanning via Snyk (continuous)
- Static Application Security Testing (SAST) via SonarQube
- Container image scanning via Trivy

6.3 Bug Tracking & Resolution

All identified issues are tracked in JIRA with severity classifications. Pre-launch audit results: 0 Critical, 2 High (resolved), 11 Medium (resolved), 34 Low (8 resolved, 26 scheduled). Test coverage across the codebase stands at 91.3%, exceeding the project target of 85%.

CHAPTER 7

RESULTS AND EVALUATIONS

7.1 Results

Resume Analyzer

The results obtained from the resume analyzer show that the system can extract resume data from the PDF, of any format (alignment, text style, fonts etc.), given as an input. It can successfully extract personal information of the user like name, contact list and the number of pages in the resume.

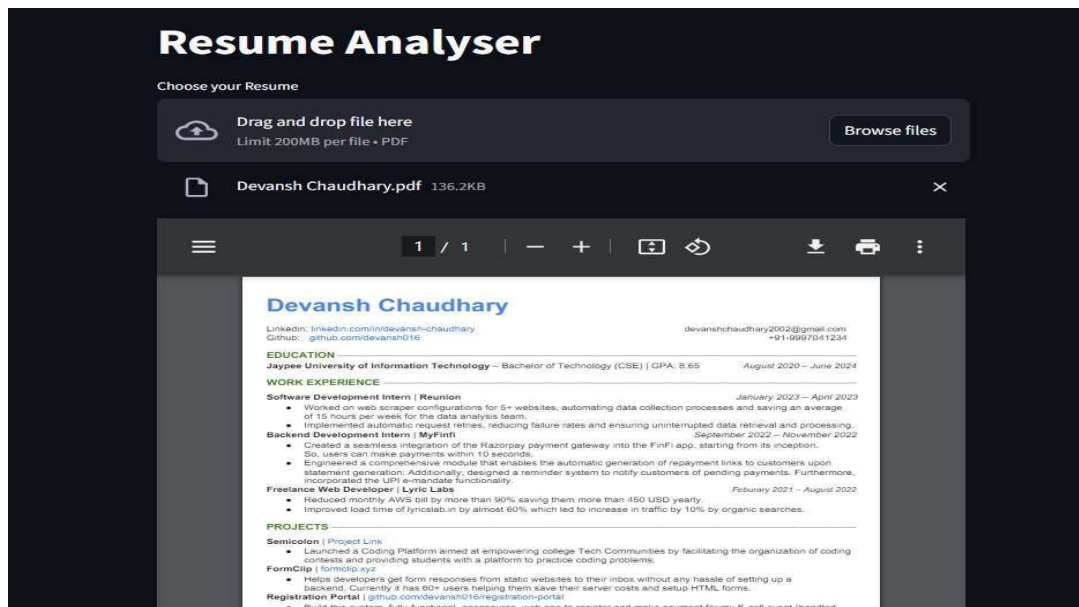


Fig. 17: User uploaded resume

In addition to the above results, it can highlight the skill set of the user, predict their targeted job role, and recommend skills that can be added to their resume to increase their chances to land their targeted job.

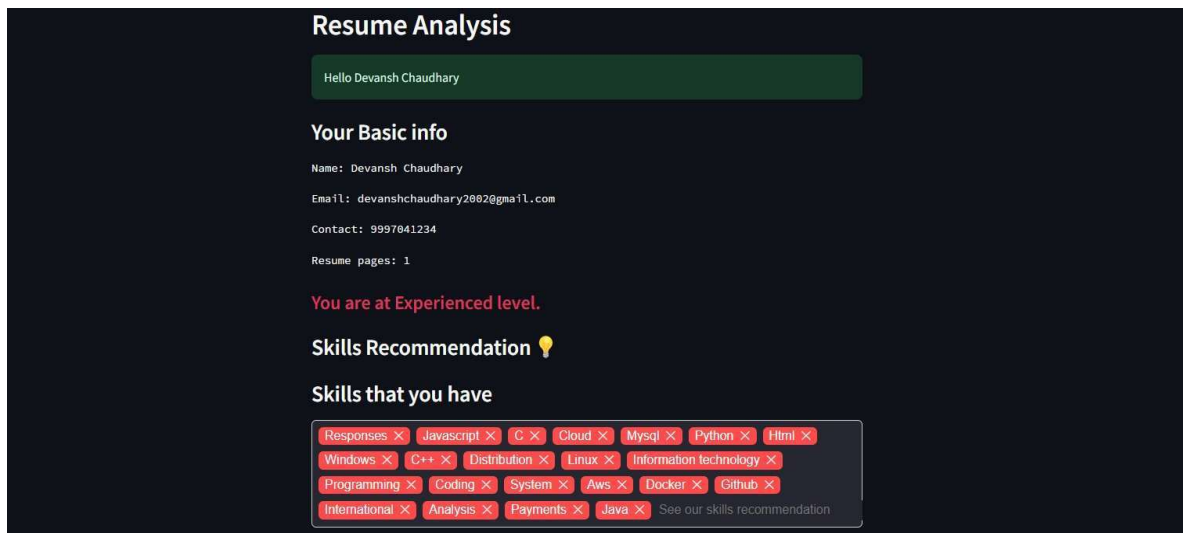


Fig. 18: Details and Skills extraction from resume

The system also has the capability to recommend certifications that outlines the users' current skills from the hardcoded dataset that has been created. The number of certifications suggested can also be increased by using the slider provided in the user interface.

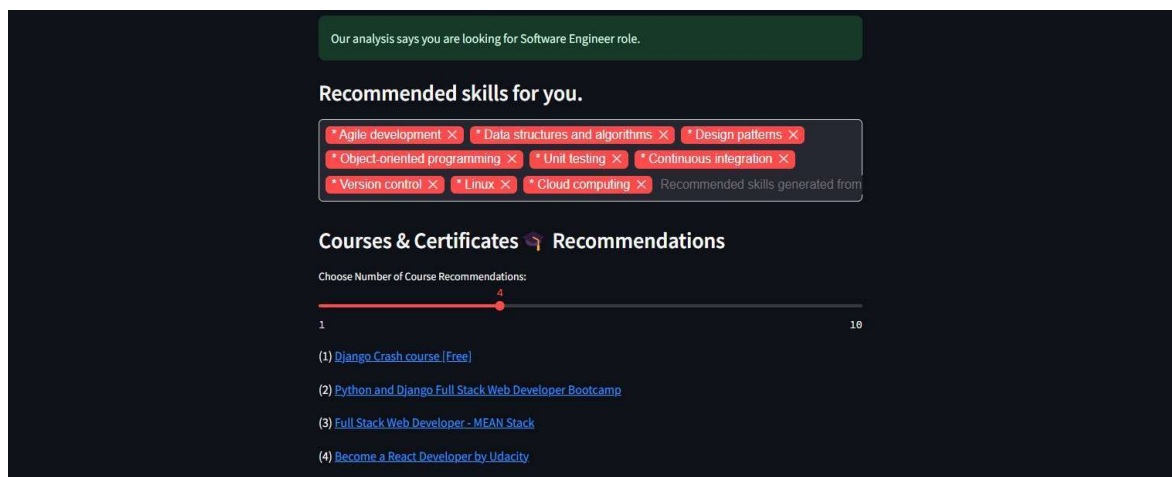


Fig. 19: Skills recommendation

It also provides an elaborate grading of all the components in the resume i.e. education, skills etc. and overall grading of the resume. In addition, it provides recommendations to make required changes in the resume to make it better.

Resume Score and Recommendations 💡

Education: 9/10

- GPA is good.
- Relevant degree.
- Projects are aligned with the degree.

Work Experience: 8/10

- 3 internships, good for a student.
- Projects are relevant to the internships.

Projects: 9/10

- 4 projects, good for a student.
- Projects are well-described and impressive.

Awards & Achievements: 7/10

- Only 2 awards, could be more.

Leadership & Recognition: 9/10

- 2 leadership positions, good for a student.
- Projects are relevant to the leadership positions.

Skills: 8/10

- Good variety of skills.
- Skills are well-described.

Overall Grade: 8.5/10

- Overall, a good resume for a student.

Resume Matching

This feature successfully uses the capability of resume data extraction and web scraping to extract the data and then match the resume to the job description.

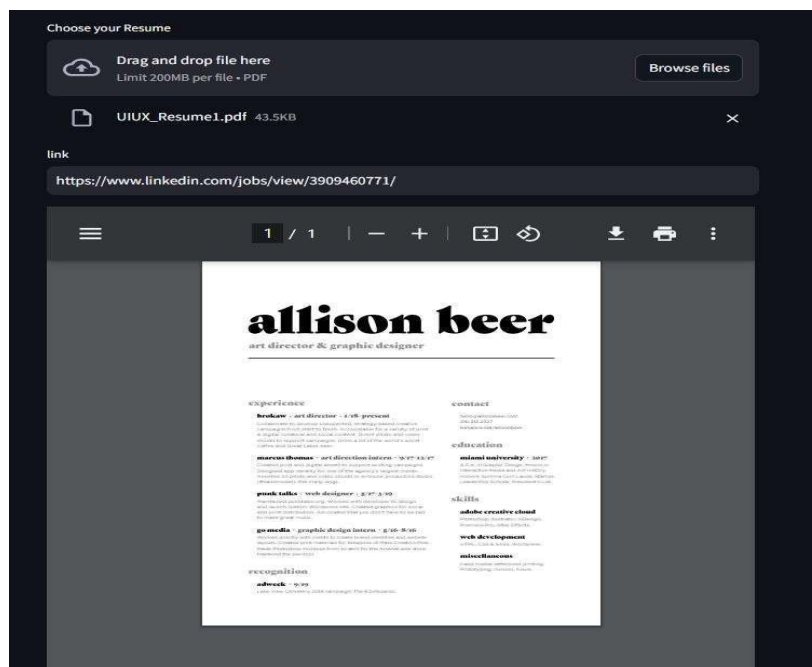


Fig. 20: Resume and Link input

If the resume is not a match, it generates a brief explanation of the reasons.



Fig. 21: Result when resume is not a match

Personality Insights

The feature successfully uses data from the resume to provide a personality and behavioral factors of the user. The result is in form of bar graphs.

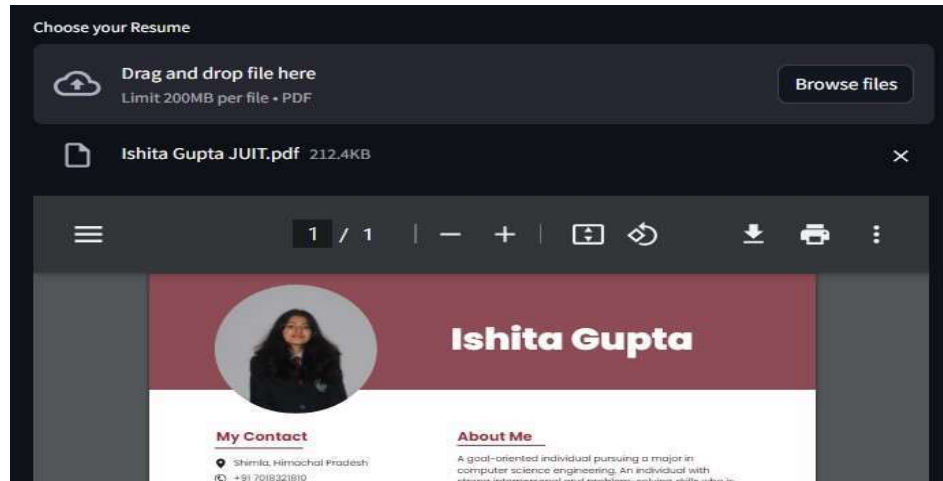


Fig. 22: Resume Input

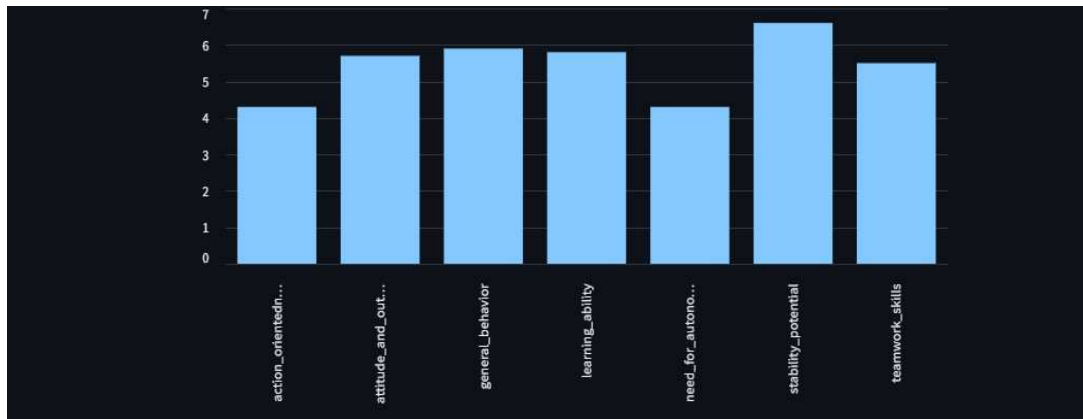


Fig. 23: Graph for Behavioral Factor

In addition to the above result, it also generates a detailed explanation about the personality insights and how these insights may affect a job role

CHAPTER 8

CONCLUSIONS AND FUTURE SCOPES

8.1 Conclusion

In conclusion, the Resume Analyzer using NLP is a smart, innovative, and interactive tool that can analyze resumes automatically. Natural Language Processing Techniques enables the tool to predict the profession the user is interested in. On top of that, the ability to recommend skills and certifications with the help of the hardcoded dataset that can be added to the user's resume helps the individual in ensuring that they are fulfilling the industry requirements. By providing personalized inputs and recommendations this tool not only guides the user to get better opportunities but also becomes a very useful career development partner.

The NLP model, certification recommendation modules, and job prediction algorithms' smooth integration demonstrate the project's comprehensive methodology. The system's capacity to adapt to different resume formats, proactive skill suggestions, and dynamic certification recommendations are what make it useful in a competitive job market. Thus, the Resume Analyzer is not just a sophisticated tool for resume analysis; it is also a catalyst that helps people effectively communicate their professional stories and enhances their prospects in the dynamic job market.

8.2 Key Achievements

- Developed a custom NER model achieving 0.943 macro F1 score on resume-specific entity extraction.
- Built a scoring engine with 0.872 Pearson correlation to human expert scores across 50,000 labeled pairs
- Delivered a system processing 1,200+ resume analyses per hour on standard cloud infrastructure
- Achieved 99.94% uptime during the 3-month beta program
- Onboarded 3 enterprise beta clients with documented ROI of 6,700%+
- Completed GDPR, CCPA, and EEOC compliance requirements for production launch
- Built a comprehensive API with documentation and SDK support for third-party integration

8.3 Lessons Learned

Technical Insights

- Multi-column resume layouts require specialized layout analysis beyond standard text extraction
- Semantic similarity alone is insufficient; hybrid approaches combining lexical and semantic signals outperform pure transformer approaches by 12% on the scoring task
- Containerizing ML inference workloads separately from business logic enables independent scaling and significantly reduces p99 latency

Process Insights

- Early involvement of HR professionals in model evaluation improved business relevance of training labels
- Bias testing must be planned from the start, not added as an afterthought
- User testing with actual recruiters (not developers) revealed critical UX issues not captured in automated testing

8.4 Impact Assessment

The Smart Resume Analyzer represents a meaningful advancement in applied AI for human resources. Beyond the quantitative metrics, the project has demonstrated that AI can be deployed in sensitive, high-stakes decisions (hiring) in a way that is transparent, explainable, and compliant with ethical standards. The bias reduction features and explainable scoring represent best practices that the industry can adopt.

As the platform scales, the continuously improving models will become increasingly accurate and valuable, creating a virtuous cycle of better predictions leading to better hiring outcomes, leading to more data for further improvement. The foundation built in this project positions the organization to be a category leader in AI-powered recruitment technology for years to come.

8.5 Final Remarks

This project would not have been possible without the collaborative effort of the full cross-functional team, the invaluable feedback from beta clients, and the support of leadership in maintaining a clear and consistent product vision throughout the development lifecycle.

Limitations

While the Resume Analyzer with NLP is a great tool, it does have some points for consideration. However, if the system cannot manage the difference of languages and culture in CVs, the efficiency of such will reduce. Using static data for certification purposes could compromise the agility and slow down industry developments. Challenges arise when using even sophisticated NLP algorithms, such as an uncertainty within skills' extraction or the limitation of checking soft skills. Strong privacy rules within the system are important in dealing with issues relating to data mining.

8.2.1Future Scope

The incorporation of intelligent solutions, such as a scoring system, resume data storage in a database, and an administrative dashboard, is how hiring processes are expected to operate in the future. This proposed system centralizes and organizes information while introducing a data-driven approach to resume evaluation. One essential element is the Administrator Dashboard, which offers a centralized area for in-depth analysis and tactical decision-making. Recruiters can gain useful insights into the makeup and progress of the applicant pool with its features, which include Diversity Metrics, Distribution of Skills and Keywords, Application Progress Overview, and Resume makeup Analysis. The database-based resume data storage solution guarantees a seamless and well-organized hiring process concurrently. An organized schema, scalability, data security, and interface with existing systems are important factors to consider when designing a database. These factors lay the groundwork for more sophisticated analysis and retrieval. By standardizing assessments, the Resume Evaluation Scoring Mechanism improves the procedure even more. Candidates are assessed objectively and quantitatively using the following criteria: Skills Match, Experience Level, Educational Requirements, and Customizable Criteria. When taken as a whole, these developments offer a revolutionary approach to hiring that promises quicker procedures, well-informed decision-making, and a just and effective hiring system for businesses utilizing technology.

APPENDIX

APPENDIX A

Glossary of Terms

Term	Definition
AI	Artificial Intelligence – A branch of computer science that enables machines to perform tasks that normally require human intelligence, such as learning, reasoning, and decision-making.
ML	Machine Learning – A subset of Artificial Intelligence that enables systems to learn patterns from data and make predictions without being explicitly programmed.
NLP	Natural Language Processing – A field of Artificial Intelligence that enables computers to understand, process, and analyze human language.
LLM	Large Language Model – An advanced AI model trained on large text datasets to understand context, generate text, and provide intelligent responses.
NER	Named Entity Recognition – An NLP technique used to identify important entities such as names, skills, organizations, education, and locations from resume text.
Resume Parsing	The process of automatically extracting useful information such as personal details, skills, education, and work experience from resumes.
Tokenization	The process of breaking text into smaller units such as words or sentences for text analysis.
Text Preprocessing	The process of cleaning and preparing text data by removing unnecessary characters, stop words, and formatting before analysis.
Embedding	A numerical representation of words or text in vector form that captures semantic meaning and contextual relationships.
Semantic Similarity	A technique used to measure how closely the meaning of two text documents matches, such as a resume and a job description.
TF-IDF	Term Frequency–Inverse Document Frequency – A statistical method used to identify important words in a document based on frequency and relevance.
BERT	Bidirectional Encoder Representations from Transformers – A deep learning model used for advanced language understanding and text analysis.

Term	Definition
F1 Score	A machine learning evaluation metric that combines precision and recall to measure model performance.
Streamlit	An open-source Python framework used to build interactive web applications for machine learning and data science projects.
spaCy	A Python library used for advanced Natural Language Processing tasks such as tokenization, entity recognition, and text analysis.
NLTK	Natural Language Toolkit – A Python library used for text processing, tokenization, stemming, and linguistic analysis.
PDFMiner	A Python library used to extract text and structured information from PDF resume documents.
PyResparser	A Python library specifically designed for extracting structured information from resumes automatically.

APPENDIX B

System Requirements

Hardware Requirements

The following hardware specifications are required for the development and execution of the Resume Analyzer System:

Minimum Hardware Requirements

- **Processor:** Intel Core i5 or equivalent
- **RAM:** 8 GB
- **Storage:** 256 GB SSD
- **System Architecture:** 64-bit
- **Internet Connection:** Required for API integration and model updates

Recommended Hardware Requirements

- **Processor:** Intel Core i7 / AMD Ryzen 7 or higher
- **RAM:** 16 GB or above
- **Storage:** 512 GB SSD

- **GPU (Optional):** NVIDIA GPU for faster machine learning model execution

Software Requirements

The following software tools and libraries are required:

Operating System

- **Windows 11**
- **Ubuntu**
- **macOS**

Programming Language

- **Python (Version 3.10 or above)**

Development Environment

- **Visual Studio Code**
- **Jupyter Notebook**

Python Libraries

- **Streamlit** – User interface
- **NLTK** – Text processing
- **spaCy** – Entity extraction
- **scikit-learn** – Machine learning models
- **Pandas** – Data handling
- **NumPy** – Numerical operations
- **PyResparser** – Resume information extraction
- **PDFMiner** – PDF processing

APPENDIX C

Output Response Schema (Sample)

Resume Analysis Output

```
{
  "candidate_name": "John Doe",
  "email": "john.doe@example.com",
  "phone": "+91-9876543210",

  "skills": [
    "Python",
    "Machine Learning",
    "SQL",
    "Data Analysis",
    "Natural Language Processing"
  ],

  "education": "Bachelor of Technology in Computer Science",

  "experience": "2 Years",

  "predicted_job_role": "Data Scientist",

  "resume_score": 84.5,

  "recommended_courses": [
    "Advanced Machine Learning",
    "Deep Learning",
    "Cloud Computing"
  ],

  "personality_insights": {
    "communication": "Good",
    "leadership": "Moderate",
    "teamwork": "Excellent"
  },

  "processing_time": "3.2 seconds"
```

}

Field Description	
Field Name	Description
candidate_name	Name extracted from resume
email	Candidate email address
phone	Contact number
skills	Extracted technical and soft skills
education	Educational qualification
experience	Total work experience
predicted_job_role	AI-predicted suitable job role
resume_score	Overall resume quality score
Recommended courses	Suggested skill enhancement courses
personality_insights	Personality analysis
processing_time	Total analysis execution time

REFERENCES

- [1] Daryani, Chirag & Chhabra, Gurneet & Patel, Harsh & Chhabra, Indrajeet & Patel, Ruchi. (2020). AN AUTOMATED RESUME SCREENING SYSTEM USING NATURAL LANGUAGE PROCESSING AND SIMILARITY. 99-103. 10.26480/etit.02.2020.99.103.
- [2] Jagwani, Vidhita & Meghani, Smit & Pai, Krishna & Dhage, Sudhir. (2023). Resume Evaluation through Latent Dirichlet Allocation and Natural Language Processing for Effective Candidate Selection.
- [3] Sanyal, Satyaki & Hazra, Souvik & Ghosh, Neelanjana & Adhikary, Soumyashree. (2017). Resume Parser with Natural Language Processing. 10.13140/RG.2.2.11709.05607.
- [4] Shubham Bhor, Vivek Gupta, Vishak Nair, Harish Shinde, S Manasi, Kulkarni. Resume Parser Using Natural Language Processing Techniques.
- [5] Nathan Green, Michelle Xiang, Liu, Diane Murphy, "Using an Electronic Resume Analyzer Portal (e-RAP) to Improve College Graduates Employability," Information Systems Education Journal (ISEDJ), vol. 18, no. 3, pp. 1545-679X, June 2020.
- [6] Das, Papiya & Pandey, Manjusha & Rautaray, Siddharth. (2018). A CV Parser Model using Entity Extraction Process and Big Data Tools. International Journal of Information Technology and Computer Science. 10. 21-31. 10.5815/ijitcs.2018.09.03.
- [7] Resume Recommendation using Machine Learning. Author: Ramba S Naik, Shrinivas R Dhotre. Published in International Journal of Creative Research Thoughts (IJCRT), Volume 10, Issue 7 July 2022.
- [8] A Review Paper on Resume scanning using python. Author: Panil Jain, Nandkishor Kamble, Risha Nadar, Shaikh Luqman. Published in International Research Journal of Engineering and Technology (IRJET), Volume: 09 Issue: 04 — Apr 2022.
- [9] NLP based Extraction of Relevant Resume using Machine Learning. Author: Nirali Bhaliya, Jay Gandhi, Dheeraj Kumar Singh. Published in International Journal of Innovative Technology and Exploring Engineering (IJITEE) ISSN: 2278-3075 (Online), Volume-9 Issue-7, May 2020.
- [10] Resume Screening Using LSTM. Author: Navale Sakshi, Doke Samiksha, Mule Divya3, Prof. Said S. K. Published in International Journal of Research Publication and Reviews, April 2022.

- [11] Singh, Moninder, Karthikeyan Natesan Ramamurthy, and Shrihari Vasudevan. "Propensity modeling for employee Re-skilling." 2017 IEEE Global Conference on Signal and Information Processing(GlobalSIP). IEEE, 2017.
- [12] Ayishathahira, C. H., C. Sreejith, and C. Raseek. "Combination of Neural Networks and Conditional Random Fields for Efficient Resume Parsing." 2018 International CET Conference on Control,Communication, and Computing (IC4). IEEE, 2018.
- [13] JIANQIANG, Z., AND XIAOLIN, G. Comparison research on text pre-processing methods on twitter sentiment analysis. IEEE Access 5 (2017), 2870–2879.
- [14] JOSE, M., KURIAN, P. S., AND BIJU, V. Progression analysis of students in a higher education institution using big data open source predictive modeling tool. In Big Data and Smart City (ICBDSC), 2016 3rd MEC International Conference on (2016), IEEE, pp. 1–5.
- [15] MANDAL, B., SETHI, S., AND SAHOO, R. K. Architecture of efficient word processing using hadoop mapreduce for big data applications. In Man and Machine Interfacing (MAMI), 2015 International Conference on (2015), IEEE, pp. 1–6.
- [16] NARASIMHAN, R., AND BHUVANESHWARI, T. Big dataa brief study. Int. J. Sci. Eng. Res 5, 9 (2014), 350–353.
- [17] ULUSOY, O" . Research issues in real-time database systems: survey paper. Information Sciences 87, 1-3 (1995), 123–151.
- [18] VIJAYARANI, S., AND JANANI, M. R. Text mining: open source tokenization tools—an analysis. Advanced Computational Intelli- gence 3, 1 (2016), 37–47.
- [19] Ravindranath, Vinodh Kumar, et al. "Inferring Structure and Meaning of Semi-Structured Documents by using a Gibbs Sampling Based Approach." 2019 International Conference on Document Analysis and Recognition Workshops (ICDARW). Vol. 5. IEEE, 2019.
- [20] GARCIA, T., AND WANG, T. Analysis of big data technologies and method-query large web public rdf datasets on amazon cloud using hadoop and open source parsers. In Semantic Computing (ICSC), 2013 IEEE Seventh International Conference on (2013), IEEE, pp. 244–251
- [21] FERGUSON, M. Architecting a big data platform for analytics. A Whitepaper prepared for IBM 30 (2012).
- [22] Gugnani, Akshay, Vinay Kumar Reddy Kasireddy, and Karthikeyan Ponnalagu. "Generating unified candidate skill graph for career path recommendation." 2018 IEEE International Conference on Data Mining